# Lab 1: Linux, C++, and Git

Environment Setup and Programming Fundamentals

Jiasheng Gu · Student ID: 24020036020

Abstract

This report documents Lab 1 of the MIT Visual Navigation for Autonomous Vehicles (VNAV) course. The lab establishes the Linux / C++ / Git working environment required for the subsequent ROS labs and practices fundamental command-line and C++ skills. An Ubuntu 20.04 system was set up with the build-essential, cmake and git toolchain. A multi-file C++ demo project was compiled and run with CMake, producing the expected “Hello world! / In library” output. Two shell exercises were completed: the wc utility reported 5 lines, 8 words and 3 non-blank lines for a test file, and the fortune program appended five proverbs to fortunes.txt. Nine C++ questions on operators, references, pointers and numeric types were traced and verified by execution. Finally, a RandomVector class was implemented that generates random doubles and prints their mean, minimum, maximum and a text histogram. All exercises ran successfully, confirming the environment is ready for the following labs.

Keywords: Ubuntu; Linux shell; Git; CMake; C++; RandomVector; VNAV.

## 1 Introduction

The VNAV labs are built on top of ROS (Robot Operating System), which runs on Ubuntu and requires day-to-day use of the Linux command line, Git for version control, and C++ for writing nodes. Lab 1 has two goals: (1) to install and configure a clean Ubuntu 20.04 environment together with the necessary toolchain, and (2) to practise the basic shell commands, Git workflow and C++ compilation that will be used in every later lab. The concrete deliverables are a set of shell exercises, answers to a series of C++ comprehension questions, and a working RandomVector C++ class.

## 2 Environment Setup

### 2.1 Ubuntu 20.04 Installation

Ubuntu 20.04 LTS (Focal Fossa) was installed either as a dual-boot system or in a virtual machine (VMware Workstation Pro is recommended for a VM; a native install is recommended for production use). The package mirror was switched to the Tsinghua mirror to speed up downloads. In a VM, a NAT virtual network with host connectivity and DHCP enabled was configured for the guest.

### 2.2 Toolchain Installation

After installation, the system was updated and the build toolchain, CMake, Git and the fortune program were installed:

```
sudo apt update
sudo apt upgrade
sudo apt install build-essential cmake git
sudo apt install fortune-mod
```

The build-essential package provides g++/gcc and make, which together with cmake form the C++ build toolchain used throughout the lab.

## 3 Exercises and Results

### 3.1 Shell Commands

Exercise 1 uses the wc utility to count lines, words and non-blank lines of a file. A test file exercise1.txt was created with five lines (two of them blank):

```
hello world
this is a test


hello again
```

```
$ wc -l < exercise1.txt        # ① number of lines
5
$ wc -w < exercise1.txt        # ② number of words
8
$ grep -c '[^[:space:]]' exercise1.txt   # ③ non-blank lines
3
```

Table 1: Output of the wc exercise.

Exercise 2 installs fortune-mod and runs fortune five times, appending each proverb to ~/vnav-personal/lab1/fortunes.txt without recreating the file:

```
fortune >  fortunes.txt   # first run (create)
fortune >> fortunes.txt   # runs 2-5 (append)
fortune >> fortunes.txt
fortune >> fortunes.txt
fortune >> fortunes.txt
Oh, wow!  Look at the moon!
The power to destroy a planet is insignificant when compared to the power of
the Force.
- Darth Vader
It is undignified for a woman to play servant to a man who is not hers.
                -- Spock, "Amok Time", stardate 3372.7
"His great aim was to escape from civilization, and, as soon as he had
money, he went to Southern California."
A real patriot is the fellow who gets a parking ticket and rejoices
that the system works.
```

Table 2: Content of fortunes.txt after appending five proverbs.

## 3.2 Git Basics

Git was used to initialise a repository, stage and commit the lab files, and inspect the history:

```
cd ~/vnav-personal/lab1
git init
git add .
git commit -m "lab1"
git status
git log --oneline
```

## 3.3 C++ Project Compilation (CMake demo)

A minimal multi-file project lab1_demo was created to practise the CMake build workflow. The project consists of a library (my_lib.hpp / my_lib.cpp) and an executable (main.cpp). The CMakeLists.txt is:

```
cmake_minimum_required(VERSION 3.10)
project(hello_world VERSION 1.0 LANGUAGES CXX)
add_library(mylib my_lib.cpp my_lib.hpp)
add_executable(main main.cpp)
target_link_libraries(main PRIVATE mylib)
```

The project was configured and built in a separate build directory:

```
mkdir build && cd build
cmake ..
make
./main
Hello world!
In library
```

Table 3: Output of the compiled demo program.

## 3.4 C++ Fundamentals

Nine comprehension questions on operators, references, pointers and numeric types were answered by tracing the code and then verified by compilation and execution. The results are summarised in Table 4.

| # | Code snippet | Output | Explanation |
|---|---|---|---|
| I-1 | j = ++i; j = i++; | i = 2, j = 1 | ++i increments before use; i++ uses then increments |
| I-2 | (i < 42) ? "a" : "b" | b | 42 < 42 is false, so the else branch is taken |

| # | Code snippet | Output | Explanation |
|---|---|---|---|
| II-1 | int& ri = i; i = 5; ri = 10; | 10 10 | ri is a reference (alias) of i |
| II-2 | *j = *j * *j; | 1764 | 42 × 42 through the pointer j |
| II-3 | *(i+2) = *(i+1) - i[3]; | 0 | 24 − 24; pointer/array arithmetic |
| II-4 | void reset(int &i){i=0;} reset(j); | 0 | pass-by-reference modifies the caller's j |
| III-1 | int i; i = 3.14; | 3 | double is truncated toward zero |
| III-2 | unsigned char c = -1; | 255 | -1 wraps modulo 256 in 8-bit unsigned |
| III-3 | if (i) { i = 0; } else { i = 43; } | 0 | 42 is non-zero (truthy), so the if branch runs |

Table 4: Answers to the C++ fundamentals questions.

### 3.5 RandomVector Class

The final exercise implements a RandomVector class that abstracts a vector of doubles. The required interface is declared in random_vector.h:

```
#pragma once
#include <vector>

class RandomVector {
public:
    RandomVector(int size, double max_val = 1.0);
    double mean();
    double max();
    double min();
    void print();
    void printHistogram(int bins);
private:
    std::vector<double> data_;
};
```

The constructor fills the vector with size uniformly-distributed doubles in [0, max_val] using . The methods mean(), max() and min() are implemented with hand-written loops (the header is deliberately avoided, as required). printHistogram(bins) divides [min, max] into bins equal intervals, counts the values in each bin, and prints one asterisk per value, separated by spaces.

Compiling with g++ -std=c++11 -Wall -pedantic and running main.cpp (20 values in [0,1], 20 bins) gives:

```
0.907371 0.274055 0.210187 0.303275 0.0257569 0.622905 0.198486 0.457286 0.8043
0.785231 0.812303 0.607174 0.289318 0.757929 0.34263 0.654969 0.906692 0.677716
0.47415 0.571937
Mean: 0.534184
Min: 0.0257569
Max: 0.907371
Histogram: *   * * ** * *  * *  * ** **  * ***  **
```

Table 5: One run of the RandomVector program.

## 4 Discussion

The shell exercises reinforce two ideas that matter in robotics work: (1) wc combined with grep shows how small command-line tools can be composed to inspect files quickly, and (2) the difference between > (overwrite) and >> (append) is essential when accumulating results such as log files. The CMake demo demonstrates the standard out-of-source build (mkdir build && cd build && cmake .. && make), which keeps generated files separate from source files.

The C++ questions highlight several error-prone details. The prefix and postfix increment operators (++i vs i++) differ in evaluation order; a reference is an alias while a pointer stores an address; integer types have platform-dependent widths and signedness, and assigning a floating-point value to an int truncates it. The unsigned char c = -1 example (yielding 255) illustrates modulo wrap-around of unsigned types. The RandomVector exercise demonstrates that mean/min/max are straightforward to implement by hand, and

that a text histogram is a compact way to visualise a distribution without any plotting library.

A limitation of the current RandomVector implementation is that random_device/ mt19937 is not seeded deterministically, so every run produces a different sequence; this is acceptable for the exercise but a fixed seed would be needed for reproducible tests. The histogram bin edges could also be exposed more clearly if the exercise required quantitative output.

## 5 Conclusion

Lab 1 established the Ubuntu 20.04 / build-essential / cmake / git toolchain and completed the shell, Git, C++ and RandomVector exercises successfully. The environment and workflows verified in this lab—package installation, source control, CMake builds, and C++ programming—are the foundation for the ROS-based experiments that follow.

## References

[1] MIT VNAV Lab 1 handouts, 2023. https://vnav.mit.edu/labs_2023/lab1/

[2] Lab 1: Linux, C, Git (course notes). https://spurious-cornflower-507.notion.site/

[3] Ubuntu 20.04 LTS (Focal Fossa) release. https://releases.ubuntu.com/20.04/

[4] OpenCV Documentation. https://docs.opencv.org/